Machine Learning HW05

Student ID: 313832010. Name: 陳祖佑

Preface

The screenshots was taken from Google Colab (jupyter notebook, .ipynb).

1. Gaussian Process

1.1 Code

1.1.1 Data Load In (Used in Task1, Task2)

```
import numpy as np
from scipy.optimize import minimize
data = np.loadtxt("input.data")
X = data[:,0]
Y = data[:,1]
X_Draw = np.arange(-60,60,0.1)
```

data.shape = (34,2) #34 pairs of (x, y)

For train, X = {x|data}, Y = {y|data}

For draw (predict the curve) → -60 ~ 60.

1.1.2 Kernel Function (Rational Quadratic Kernel), (Used in Task1, Task2)

```
beta = 5
sigma_n = 1/beta
def kernel(xa, xb, sigma, length, alpha):
        xa = xa.reshape(-1, 1)
        xb = xb.reshape(1, -1)
        d2 = (xa - xb)**2
        val = sigma**2 * (1 + d2/(2*alpha*length**2))**(-alpha)
        return val
```

$$Kernel(x,x') = \sigma_f^2(1+\frac{(x-x')^2}{2\alpha l^2})^{-\alpha}$$

1.1.3 Posterior Calculation & Draw Function

```
def posterior_predict_and_draw(X,Y,X_Draw,sigma,length,alpha,noise):
    K = kernel(X, X, sigma, length, alpha)
    K_s = kernel(X,X_Draw, sigma, length, alpha)
    K_ss = kernel(X_Draw, X_Draw, sigma, length, alpha)

    C = K + noise * np.eye(len(X))
    L = np.linalg.cholesky(C) #C=L*L.T
    v = np.linalg.solve(L, Y) #Lv=Y
    w = np.linalg.solve(L.T, v) #L.T * w = v

    A = np.linalg.solve(L, K_s)          # shape: N×M
    B = np.linalg.solve(L.T, A)
    Posterior_Mean = K_s.T @ w
    Posterior_Var = np.diag( K_ss - K_s.T @ B )

    plt.scatter(X, Y)
    plt.plot(X_Draw, Posterior_Mean, label='Predictive Mean')
    std = np.sqrt(Posterior_Var)
    upper = Posterior_Mean + 1.96 * std
    lower = Posterior_Mean - 1.96 * std
    plt.fill_between(X_Draw, lower, upper, alpha=0.3, label='95% Confidence Interval')
    plt.legend()
    plt.show()

posterior_predict_and_draw(X,Y,X_Draw,sigma0,length0,alpha0,noise0)
```

In posterior_predict_and_draw

1. Calculate Kernels and built covariance matrices.
2. Calculate Kernels for prediction and draw.
3. Calculate covariance matrix, $C = K + \sigma_n^2 I$.
4. For plotting, calculate posterior mean and variance.
5. Plot posterior curve (with 95% confidence interval)

Note: 95% confidence interval actually stands for ± 1.96 times of STD.

Called the function to draw initial setting/guessing. (Task1)

1.1.4 NLL (Negative marginal Log-Likelihood) (for Task2 training)

```
def NLL(theta):
        sigma = np.exp(theta[0])
        length = np.exp(theta[1])
        alpha = np.exp(theta[2])
        noise = np.exp(theta[3])
        K = kernel(X, X, sigma, length, alpha)
        C = K + noise * np.eye(len(X))


        L = np.linalg.cholesky(C)  #C=L*L.T
        v = np.linalg.solve(L, Y)  #Lv=Y
        w = np.linalg.solve(L.T, v)  #L.T * w = v
        log_C = 2 * np.sum(np.log(np.diag(L)))
        N = len(X)
        nll = 0.5 * (Y @ w + log_C + N * np.log(2*np.pi))
        return nll
```

In NLL: (Used in Task2 Training)

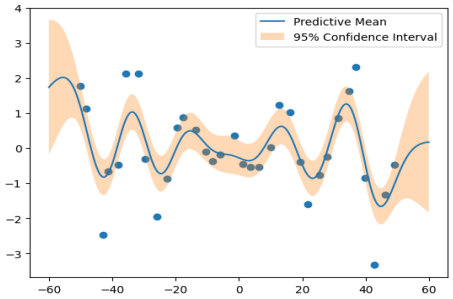
1. Extract Parameters.
2. Calculate Kernel
3. Calculate Covariance Matrix with Kernel.
4. $NLL = \frac{1}{2}(y^T C^{-1} y + \log|C| + nlog(2\pi))$

1.2 Experiments

1.2.1 Initial Setting / Guessing

```
sigma0 = np.std(Y)
length0 = 10
alpha0 = 1
noise0 = 1/beta
```

1.2.2 Task 1 Result



Put parameters in 1.2.1 into posterior_predict_and_draw function, as shown in 1.1.3's last line.

1.2.3 Train for result and output result

```
theta0 = [np.log(sigma0), np.log(length0), np.log(alpha0), np.log(noise0)]
result = minimize(NLL, theta0, method='L-BFGS-B')
```

```
print(result)
```

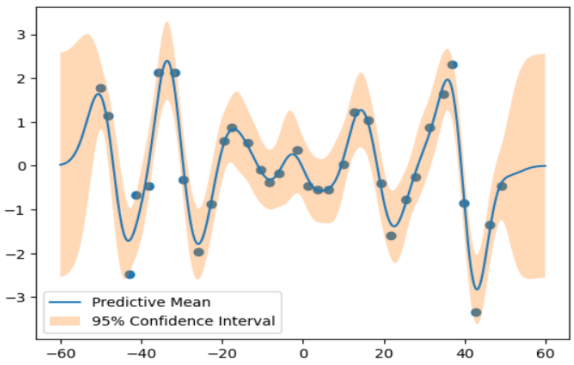
```
  message: CONVERGENCE: RELATIVE REDUCTION OF F <= FACTR*EPSMCH
  success: True
   status: 0
      fun: 50.63107688336776
        x: [ 2.659e-01  1.225e+00  8.755e+00 -1.441e+00]
      nit: 29
      jac: [ 3.858e-04  6.473e-04 -2.537e-04 -5.691e-04]
     nfev: 275
     njev: 55
 hess_inv: <4x4 LbfgsInvHessProduct with dtype=float64>
```

1. Train with initial setting (defined in 1.2.1)
2. Print out result: result.x contains trained parameters.
3. Trained-parameters: (noise: σ_n^2)

$\sigma = 0.2659,\ l = 1.225,\ \alpha = 8.755,\ \sigma_n^2 = -1.441$

1.2.4 Draw trained (Task2) result

```
Train_Result = np.array(result.x)
Train_Result = np.exp(Train_Result)
sigma, length, alpha, noise = Train_Result
posterior_predict_and_draw(X, Y, X_Draw, sigma, length, alpha, noise)
```



Plot final result.

Compared with Task1's Result, more data points stays in 95% Confidence Interval.

1.3 Observation and Discussion

In Task1's Result, the result was not that bad, it still follows to the data's trend. That's because GP is a Bayesian model, once it has kernel, it can make regression, predict uncertainty. Most Importantly, it can use combination of train and prediction to calculate covariance, posterior mean, posterior variance.

After minimize NLL, the parameters (theta) optimized. The optimized model better calibrates the smoothness (via the length scale) and the noise level, results in minor of total distance from target. Also, 95% theory works in optimized model.

2. SVM on MNIST

Note: Due to libsvm version incompatible, I use libsvm-official instead, which work normally.

```
[1] ✓ 13 秒
pip install -U libsvm-official

Collecting libsvm-official
  Downloading libsvm-official-3.36.0.tar.gz (40 kB)
  Preparing metadata (setup.py) ... done
Requirement already satisfied: scipy in /usr/local/lib/python3.12
Requirement already satisfied: numpy<2.6,>=1.25.2 in /usr/local/l
Building wheels for collected packages: libsvm-official
  Building wheel for libsvm-official (setup.py) ... done
  Created wheel for libsvm-official: filename=libsvm_official-3.3
  Stored in directory: /root/.cache/pip/wheels/df/65/4b/c3cdece6e
Successfully built libsvm-official
Installing collected packages: libsvm-official
Successfully installed libsvm-official-3.36.0
```

```
[2] ✓ 2 秒
import numpy as np
import pandas as pd
from libsvm.svmutil import svm_train, svm_predict
```

Detailed in Discussion.

2.1 Code

2.1.1 Data Load in and Preprocessing

```
df = pd.read_csv("X_train.csv", header=None)
X_train = np.array(df)
df = pd.read_csv("Y_train.csv", header=None)
Y_train = np.array(df)
Y_train = Y_train.reshape(-1)
df = pd.read_csv("X_test.csv", header=None)
X_test = np.array(df)
df = pd.read_csv("Y_test.csv", header=None)
Y_test = np.array(df)
Y_test = Y_test.reshape(-1)
```

Reshape for libsvm.

(libsvm requires single dimension Y, sparse matrix recorded X and simple command as input for training.)

```
def to_libsvm_format(X):
        data = []
        for i in range(X.shape[0]):
                d = {}
                for j in range(X.shape[1]):
                        if X[i, j] != 0:
                                d[j+1] = float(X[i, j]) #有值才紀錄(稀疏矩陣)
                data.append(d)
        return data

X_train_libsvm = to_libsvm_format(X_train)
X_test_libsvm = to_libsvm_format(X_test)
Y_train = [int(y) for y in Y_train]
Y_test = [int(y) for y in Y_test]
```

2.1.2 Linear kernel, Polynomial kernel, RBF kernel (Task1)

Linear SVM

```
svm_model_linear = svm_train(Y_train, X_train_libsvm, '-s 0 -t 0 -c 1')
pred_label, pred_acc, pred_vals = svm_predict(Y_test, X_test_libsvm, svm_model_linear)
```

```
Accuracy = 95.08% (2377/2500) (classification)
```

Polynomial SVM

```
gamma = 1/784
svm_model_poly = svm_train(Y_train, X_train_libsvm, '-t 1 -d 3 -g %f -r 1 -c 1'%gamma)
pred_label, pred_acc, pred_vals = svm_predict(Y_test, X_test_libsvm, svm_model_poly)
```

```
Accuracy = 95.76% (2394/2500) (classification)
```

RBF SVM

```
svm_model_RBF = svm_train(Y_train, X_train_libsvm, '-s 0 -t 2 -g 0.001 -c 1')
pred_label, pred_acc, pred_vals = svm_predict(Y_test, X_test_libsvm, svm_model_RBF)
```

```
Accuracy = 95.24% (2381/2500) (classification)
```

-t	Types of kernel.
-c	Tolerance of error.
-g	Gamma.
-d	Dimension of Polynomial

2.1.3 C-SVC (Task 2)

```
import io
import contextlib

def svm_train_SVC(Y_train, X_train_libsvm, C, gamma):
    buffer = io.StringIO()

    param = '-s 0 -t 2 -c %f -g %f -v 5'%(C, gamma)

    with contextlib.redirect_stdout(buffer):
            svm_train(Y_train, X_train_libsvm, param)

    output = buffer.getvalue()
    Record = False
    Acc = ''
    for i in range(len(output)):
        if Record==True:
            Acc+=output[i]
        elif output[i]=='=' or output[i]=='%':
            if Record==True:
                Record=False
            else:
                Record=True
    Acc = float(Acc[:-2])/100
    print("C: %f, gamma: %f, Acc: %f"%(C, gamma, Acc))

    return Acc
```

Single Round of SVM training:

1. Changeable training params.
2. Train out performance grade.
3. Strip out performance from string and return.

(output contains whole output information string.)

```
C_List = [0.01, 0.1, 1, 10, 100]
Gamma_List = [0.0001, 0.001, 0.01, 0.1]
Highest_Acc = 0
Best_C = 0
Best_Gamma = 0

for i in range(len(C_List)):
    for j in range(len(Gamma_List)):
        Acc = svm_train_SVC(Y_train, X_train_libsvm, C_List[i], Gamma_List[j])
        if Acc > Highest_Acc:
            Highest_Acc = Acc
            Best_C = C_List[i]
            Best_Gamma = Gamma_List[j]

best_param = '-s 0 -t 2 -c %f -g %f' % (Best_C, Best_Gamma)
Final_Model = svm_train(Y_train, X_train_libsvm, best_param)
pred_label, pred_acc, pred_vals = svm_predict(Y_test, X_test_libsvm, Final_Model)

print("Final Test Accuracy =", pred_acc)
```

1. Take different combination of C and Gamma into training, finding best parameters setting.
2. After finding best parameters, train out final model and using test set to test performance.

2.1.4 Combinational Kernel (Task 3)

```
def Linear_RBF_Combinational_Kernel(x, z, alpha, gamma):
    linear_K = alpha * (x.T @ z)
    RBF_K = (1-alpha) * np.exp(-gamma * np.linalg.norm(x-z)**2)
    return linear_K + RBF_K
```

```
#Convert into Kernel
K_train = np.zeros((len(X_train), len(X_train)+1))
alpha = 0.7
gamma = 1/784
for i in range(len(X_train)):
        K_train[i][0] = i+1
        for j in range(len(X_train)):
                K_train[i][j+1] = Linear_RBF_Combinational_Kernel(X_train[i], X_train[j], alpha, gamma)

model = svm_train(Y_train, K_train.tolist(), '-s 0 -t 4 -c 1')
K_test = np.zeros((len(X_test), len(X_train)+1))
for i in range(len(X_test)):
        K_test[i][0] = i+1
        for j in range(len(X_train)):
                K_test[i][j+1] = Linear_RBF_Combinational_Kernel(X_test[i], X_train[j], alpha, gamma)

pred_label, pred_acc, pred_vals = svm_predict(Y_test, K_test.tolist(), model)
```

```
Accuracy = 95.24% (2381/2500) (classification)
```

Linear Kernel definition: $K_{Linear} = XX^T$.

RBF Kernel definition: $K_{RBF,ij} = e^{(-\gamma||x_i - x_j||^2)}$.

Combinational Kernel: (For each (i,j) picture.)

$$\alpha \cdot (K_{Linear,ij}) + (1-\alpha) \cdot (K_{RBF,ij})$$

Process:

1. Built train kernel and train out a model.
2. Built train-predict kernel and used trained model for performance predict.

2.2 Experiments

2.2.1 Task 1

Using test dataset for accuracy prediction, which contains 2500 samples. For execution time estimation, I used **timeit**.

	Test Accuracy	Test Correct #	Train Time
Linear Kernel	95.08%	2377 / 2500	2.33s
Polynomial Kernel	95.76%	2394 / 2500	4.91s
RBF Kernel	95.24%	2381 / 2500	4.94s

2.2.2 Task 2

Finding best parameter setting among **C_List** and **Gamma_List**.

C_List = {0.01, 0.1, 1, 10, 100}

Gamma_List = {0.0001, 0.001, 0.01, 0.1}

```
C: 0.010000, gamma: 0.000100, Acc: 0.796400
C: 0.010000, gamma: 0.001000, Acc: 0.808400
C: 0.010000, gamma: 0.010000, Acc: 0.924200
C: 0.010000, gamma: 0.100000, Acc: 0.484600
C: 0.100000, gamma: 0.000100, Acc: 0.796400
C: 0.100000, gamma: 0.001000, Acc: 0.925000
C: 0.100000, gamma: 0.010000, Acc: 0.963200
C: 0.100000, gamma: 0.100000, Acc: 0.540000
C: 1.000000, gamma: 0.000100, Acc: 0.925800
C: 1.000000, gamma: 0.001000, Acc: 0.961800
C: 1.000000, gamma: 0.010000, Acc: 0.978000
C: 1.000000, gamma: 0.100000, Acc: 0.918800
C: 10.000000, gamma: 0.000100, Acc: 0.961200
C: 10.000000, gamma: 0.001000, Acc: 0.972400
C: 10.000000, gamma: 0.010000, Acc: 0.982400
C: 10.000000, gamma: 0.100000, Acc: 0.922200
C: 100.000000, gamma: 0.000100, Acc: 0.969000
C: 100.000000, gamma: 0.001000, Acc: 0.974800
C: 100.000000, gamma: 0.010000, Acc: 0.982600
C: 100.000000, gamma: 0.100000, Acc: 0.923200
Accuracy = 98.16% (2454/2500) (classification)
Final Test Accuracy = (98.16, 0.0504, 0.9749455965787434)
```

1. Finding best parameter pair through grid (cross) training.
2. Use optimal parameter pair to train final model.
3. Use the model to predict test accuracy.

Train detail: (Vertical stands for Gamma value; Horizontal stands for C value)

Acc Ratio	0.01	0.1	1	10	100
0.0001	0.7964	0.7964	0.9258	0.9612	0.969
0.001	0.8084	0.925	0.9618	0.9724	0.9748
0.01	0.9242	0.9632	0.978	0.9824	0.9826
0.1	0.4846	0.54	0.9188	0.9222	0.9232

Train Time	0.01	0.1	1	10	100
0.0001	116.4148	115.1439	67.8755	26.42753	12.8495
0.001	116.6588	70.86731	26.69939	13.60297	11.08596
0.01	99.08972	35.8584	16.51209	15.4486	15.44969
0.1	115.9922	110.7818	113.0356	112.3496	110.2121

Best performance case (Acc Ratio=98.26%), trained with C = 100 and Gamma = 0.01.

Using optimal parameter pair to train the model, results in 98.16% test accuracy, which is higher than any of kernel in Task 1.

2.2.3 Task 3

Different ratio between Linear and RBF results in slightly different.

As defined in 2.1.4: $Kernel = \alpha \times Linear\ Kernel + (1 - \alpha) \times RBF\ Kernel$

	Test Accuracy	Test Correct #
α =0.7	95.24%	(2381/2500)
α =0.5	95.52%	(2388/2500)
α =0.3	95.84%	(2396/2500)

Train with Hybrid kernel produces better testing performance than Linear Kernel (95.08%) or RBF Kernel (95.24%).

2.3 Observation and Discussion

2.3.1 Discussion: libsvm and libsvm-official:

The version incompatible issue happens when libsvm calling scipy's numpy, which invoke: AttributeError: Module 'scipy' has no attribute 'ndarray'. In newer version of scipy, scipy didn't support numpy any more.

2.3.2 Experimental Result Observation:

In task2, compare best case (Acc Ratio=98.26%) to worst case (Acc Ratio=48.46%), the train time cost about only 13%. Besides, setting γ =0.01 produces better performance in all kind of C.

In task3, the calculation of kernel takes about 95% of whole time. Since I calculate picture to picture, for training kernel it takes 5000×5000 times of calculation, while test kernel takes 2500×5000 times of calculation. So the whole process takes about 10 minutes with Google Colab CPU.